

附表 A-2 gdb 常见命令

简 写	全 称	备 注
l	list	显示指定行号或者指定函数附近的源代码
b	break	在指定行号或者指定函数开头处设置断点。如 b main
r	run	运行程序，直到程序结束或者遇到断点而停下
c	continue	在程序中断后继续执行程序，直到程序结束或者遇到断点而停下。注意在程序开始执行前只能用 r，不能用 c
n	next	执行一条语句。如果有函数调用，则把它作为一个整体
s	step	执行一条语句。如果有函数调用，则进入函数内部
u	until	执行到指定行号或者指定函数的开头
p	print	显示变量或表达式的值
disp	display	把一个表达式设置为 display，当程序每次停下来时都会显示其值
cl	clear	取消断点，和 b 的格式相同。如果该位置有多个断点，将同时取消
i	info	显示各种信息。如 i b 显示所有断点，i disp 显示 display，而 i lo 显示所有局部变量

如果对上述解释有疑问，可输入 help 以获得详尽的帮助信息。

A.3.4 gdb 的高级功能

gdb 的功能远不止刚才所讲述的那些。尽管很多功能是专为系统级调试所设，但还有很多功能也能为算法程序的调试带来很大方便。

首先是栈帧的相关命令，其中最常用的是 bt，其他命令可以通过 help stack 来学习。接下来是断点控制命令。commands (comm) 命令可以指定在某个断点处停下来后所执行的 gdb 命令，ignore (ig) 命令可以让断点在前 count 次到达时都不停下来，而 condition 则可以给断点加一个条件。例如，在下面的循环中：

```
10 for(i = 0; i < n; i++)
11 printf("%d\n", i);
```

首先用 b 11 设置断点（假设编号为 2），然后用 cond 2 i==5 让该断点仅当 i=5 时有效。这样的条件断点在进行细致的调试时往往很有用。

另外，gdb 还支持一种特殊的断点——watchpoint。例如，watch a（简写为 wa a）可以在变量 a 修改时停下，并显示出修改前后的变量值，而 awatch a（简写为 aw a）则是在变量被读写时都会停下来。类似地，rwatch a（rw a）则是在变量被读时停下。

最后需要说明的是，gdb 中可以自由调用函数（不管是源程序中新定义的函数还是库函数）。第一种方法是用 call 命令。例如，如果想给包含 10 个元素的数组 a 排序，可以像这样直接调用 STL 中的排序函数 call sort(a, a+10)。

遗憾的是，如果真的做过这个实验，会发现刚才所说完全是骗人的。gdb 会显示不存在函数 sort。怎么会这样呢？如果学过宏和内联函数就会知道，很多看起来是函数的却不一定是函数，或者说，不一定是调试器识别的函数。为了在 gdb 中调用 sort，可以将它打包：

```
void mysort(int*p, int*q)
{
    sort(p, q);
}
```

这样，就可以用 `call mysort(a, a+10)` 来给数组 `a` 排序了。`print`、`condition` 和 `display` 命令都可以像这样使用 C/C++ 函数。例如，可以用 `p rand()` 来输出一个随机数，或是专门编写一个打印二叉树的函数，然后在 `print` 或者 `display` 命令中使用它，还可以编写一个返回 `bool` 值的函数，并作为断点的条件。

至此是不是觉得 `gdb` 很强大呢？注意，过分地依赖于 `gdb` 的调试功能让敏锐的直觉变得迟钝。事实上，笔者建议读者尽量只使用 A.3.3 节提到的基本功能，甚至尽量不要使用 `gdb`——用输出中间变量的方法，加上直觉和经验来调试算法程序。如果是这样，编程速度和准确性将大大提高。

A.4 浅谈 IDE

所谓 IDE，是指集成开发环境（Integrated Development Environment）。顾名思义，开发程序所用到的各种功能都应该被集成到 IDE 中，包括编辑（edit）、编译（compile）、运行（run）、调试（debug）等。但工具始终总是工具，读者必须懂得如何使用它，才能发挥出它的最大威力。

可以用来编写 C/C++ 程序的 IDE 有很多，如 Linux 下的 Anjuta，Windows 下的 Dev-Cpp，以及跨平台的 Eclipse 和 Code::Blocks，还有一些强大的通用编辑器也可以用来编写 C/C++ 程序，如 `vi`、`emacs`、`EditPlus` 等。

也许和很多读者所期望的不同，笔者在这里不打算介绍任何一个 IDE。事实上，如果读者对本章所介绍的命令行、脚本、编译选项和 `gdb` 都能很好地掌握，IDE 是很容易学习的——只需要熟悉它的编辑特色（语法高亮、代码折叠、查找与替换和代码补全等）和常用快捷键即可。

多数 IDE 会引入“工程”的概念，所以读者需要花一点时间来掌握工程的基本知识。例如，在编写算法程序时，工程类别需要的是命令程序（console application），而不是图形界面程序（GUI application）或其他。如果熟练掌握了 `gcc` 编译参数和 `gdb` 的常见命令，在 IDE 下编译和调试会更容易。

主要参考书目

- [1] Thomas H.Cormen,Charles E.Leiserson,Ronald L.Rivest,Clifford Stein.Introduction to Algorithms,Second Edition,The MIT Press,2001
- [2] Jon Kleinberg,Éva Tardos.Algorithm Design.Addison Wesley,2005
- [3] Sanjoy Dasgupta.Christos Papadimitriou,Umesh Vazirani.Algorithms.McGraw Hill Higher Education,2006
- [4] Ronald L.Graham,Donald E.Knuth,Oren Patashnik.Concrete Mathematics.Addison-Wesley Professional,1994
- [5] Joseph O'Rourke.Computational Geometry in C,second edition,Cambridge University Press,1998
- [6] Mark de Berg,Otfried Cheong, Marc van Kreveld,Mark Overmars,Computational Geometry: Algorithms and Applications, 3rd Edition, Springer-Verlag Berlin and Heidelberg GmbH & Co. K,2008
- [7] G. Polya. How to Solve It: A New Aspect of Mathematical Method. Princeton University Press,2nd Edition,1971
- [8] Philip Schneider, David H. Eberly. Geometric Tools for Computer Graphics. Morgan Kaufmann,2002
- [9] 周培德. 计算几何——算法分析与设计. 北京: 清华大学出版社, 2000
- [10] 中国计算机学会(执行主编王宏). 全国信息学奥林匹克年鉴. 河南: 中原出版传媒集团, 2006—2010
- [11] 中国计算机学会. IOI 国家集训队资料, 2002—2010

丛书介绍

算法在计算机科学乃至整个科学界的作用日益明显。它们不仅具有重要的理论意义，而且解决了生产生活中的很多实际问题。程序设计竞赛就是这样一类以算法为核心但是偏重实用性的比赛。随着各类比赛规模的逐渐扩大，程序设计竞赛在各高校、IT公司和其他社会各界中越来越受到认可和重视。很多研究工作者和从事IT行业的人尽管不参加这类竞赛，但也希望具有这方面的能力，受到这方面的专业训练。

本丛书的前身是之前的同名图书《算法艺术与信息学竞赛》。近年来，更多的人加入到参赛、命题和组织的队伍中来，各类竞赛的参赛和命题水平也有了长足的进步。作者深知当年的经典之作开始显得题目陈旧，知识的广度和深度也无法达到当今高水平比赛的要求了。因此，将原书的内容扩充、完善，以丛书的形式依次展现给读者。这些书循序渐进，从零语言基础开始讲起，直到超越竞赛本身，真正把算法当成“艺术”。

- 《算法竞赛入门经典》
- 《算法竞赛入门经典——训练指南》
- 《算法竞赛入门经典（第2版）》

代码下载: <http://code.google.com/p/aoapc-book/>
<http://www.tup.com.cn>

清华大学出版社数字出版网站

WQBook  书文局泉
www.wqbook.com

ISBN 978-7-302-35628-8



9 787302 356288 >

定价: 49.80元